



BCSE498J- Project - II

CROP PRICE PREDICTION USING DEEP LEARNING FOR TAMIL NADU

Submitted in partial fulfilment of the requirements for the degree of

Bachelor of Technology

in

Computer Science and Engineering

by

22BCE0508 TANISHQ ANAND

Under the Supervision of

Prof. Manjula R

Professor Grade 1

School of Computer Science and Engineering



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

April, 2026

DECLARATION

I hereby declare that the project entitled “CROP PRICE PREDICTION USING DEEP LEARNING FOR TAMIL NADU” submitted by me, for the award of the degree of Bachelor of Technology in Computer Science and Engineering to VIT is a record of bonafide work carried out by me under the supervision of Prof. Manjula R, School of Computer Science and Engineering.

I further declare that the work reported in this project report has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Vellore

TANISHQ ANAND (22BCE0508)

Date :

CERTIFICATE

This is to certify that the project entitled “CROP PRICE PREDICTION USING DEEP LEARNING FOR TAMIL NADU” submitted by TANISHQ A School of Computer Science and Engineering, VIT, for the award of t elor of Technology in Computer Science and Engineering , is a record of work carried out by the student under my supervision during Winter Semester 2025-2026, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The project fulfills the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

Place: Vellore

Signature of the Guide

Date :

Internal Examiner

External Examiner

Head of the Department

ACKNOWLEDGEMENT

I TANISHQ ANAND (22BCE0508) am deeply grateful to the management of Vellore Institute of Technology (VIT) for providing me with the opportunity and resources to undertake this project. Their commitment to fostering a conducive learning environment has been instrumental in my academic journey. The support and infrastructure provided by VIT have enabled me to explore and develop my ideas to their fullest potential.

My sincere thanks to Dr. Jaisankar N, the Dean of the School of Computer Science and Engineering, for constant support and encouragement. The Dean's leadership and vision have greatly inspired me to strive for excellence. The dedication to academic excellence and innovation has been a constant source of motivation.

I express my profound appreciation to Dr. Boominathan P, the Head of the Department of Software Systems for insightful guidance and continuous support. The expertise and advice have been crucial in shaping the direction of my project. The commitment to fostering a collaborative and supportive atmosphere has greatly enhanced my learning experience. The constructive feedback and encouragement have been invaluable in overcoming challenges and achieving my project goals.

I am immensely thankful to my project supervisor, Prof. Manjula R, School of Computer Science and Engineering, for dedicated mentorship and invaluable feedback. The patience, knowledge, and encouragement have been pivotal in the successful completion of this project. The willingness to share expertise and provide thoughtful guidance has been instrumental in refining my ideas and methodologies. This support has not only contributed to the success of this project but has also enriched my overall academic experience.

I extend my heartfelt thanks to all for the guidance and support received throughout this endeavor.

TABLE OF CONTENTS

Executive Summary	ii
List of Tables	iii
List of Figures	iv
List of Abbreviations	v
List of Symbols	vi
1 INTRODUCTION	1
1.1 Background	1
1.2 Motivation	1
1.3 Scope of the Project	2
2 PROJECT DESCRIPTION AND GOALS	4
2.1 Literature Review	4
2.2 Research Gap	5
2.3 Objectives	5
2.4 Problem Statement	6
2.5 Project Plan	6
3 TECHNICAL SPECIFICATION	8
3.1 System Overview	8
3.2 Data Specification	8
3.3 Feature Engineering	9
3.4 Target Construction	9
3.5 Modeling Framework	9
3.6 Training and Validation Strategy	10
3.7 System Implementation	10
3.8 Hardware and Software Environment	10
4 SYSTEM DESIGN	12
4.1 System Architecture	12

4.2	Data Flow and Processing Pipeline	13
4.3	Model Design	13
4.4	Ensemble Design and Ranking Mechanism	14
4.5	Design Considerations	14
5	METHODOLOGY AND TESTING	15
5.1	Feature Engineering	15
5.1.1	Normalized Productivity Features	15
5.1.2	Yield and Area Potential	15
5.1.3	Combined Productivity Score	15
5.2	Target Construction	15
5.3	Model Training	16
5.4	Validation and Evaluation	16
5.5	Ensemble Construction	16
5.6	Ranking Mechanism	16
6	PROJECT IMPLEMENTATION	17
6.1	Data Handling and Integration	17
6.2	Feature Engineering Implementation	17
6.3	Model Implementation	17
6.4	Validation and Evaluation Implementation	18
6.5	Ensemble and Ranking Implementation	18
6.6	System Workflow Execution	19
6.7	Implementation Challenges	19
7	RESULTS AND DISCUSSION	20
7.1	Model Performance Analysis	20
7.2	Feature Contribution Analysis	20
7.3	Effect of Hyperparameter Tuning	21
7.4	Ensemble Performance	21
7.5	Ranking Behavior and Interpretation	21
7.6	Final Crop Ranking and Recommendation	22
7.7	Discussion	23
7.8	Limitations	23

8	CONCLUSION AND FUTURE WORK	24
8.1	Conclusion	24
8.2	Future Work	24

EXECUTIVE SUMMARY

This paper introduces an AI-enabled crop recommendation system for Thanjavur, which takes into account both agroclimatic appropriateness and marketing considerations to provide a more robust solution for supporting informed agricultural choices. Unlike traditional rule-based recommendations and crop selection criteria, this approach relies on a probability-based model for ranking and classifying crops.

The proposed framework uses a binary classification technique in order to determine whether the considered example corresponds to a high-yielding crop variety. As the threshold for defining high-yielding varieties, a proxy for revenue is used; it is based on the multiplication of a market price mode and average productivity levels. Contrary to maximizing price, other factors such as soil properties, weather data, crop needs, and historical productivity are taken into consideration while training models to ensure agronomical viability remains paramount.

To select the most appropriate predictive models, several families of methods such as Random Forest, Gradient Boosting, SVMs, and Logistic Regression were considered. Boosted models like XGBoost, LightGBM, and CatBoost were used where applicable. In this paper, stratified validation was conducted, focusing primarily on the calculation of precision-recall AUC scores.

In the final step, we build the ensemble model from the best-performing algorithms and derive three rankings, namely, the likelihood of good performance, the yield, and the proxy of profitability. The decoupling enables us to develop an interpretable system while considering the agricultural and economic perspectives.

As can be seen from the experimental findings, the ensemble learning method incorporating feature domains proves to deliver solid ranking outcomes. Our framework offers a scalable platform for further developments.

LIST OF TABLES

6.1 Summary of Models and Key Hyperparameters 18

7.1 Performance Comparison of Models on Holdout Set (Best Feature Set) . 20

7.2 Best Model Performance Across Feature Sets 20

7.3 Top Crops by Predicted High-Performance Probability 22

7.4 Top Crops by Yield Potential 22

7.5 Top Crops by Profit Proxy 22

LIST OF FIGURES

4.1	System Architecture	12
6.1	Workflow Diagram	19

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
AUC	Area Under the Curve
CPU	Central Processing Unit
CSV	Comma Separated Values
CV	Cross Validation
GB	Gradient Boosting
GPU	Graphics Processing Unit
LGBM	Light Gradient Boosting Machine
LR	Logistic Regression
ML	Machine Learning
PR	Precision-Recall
PR-AUC	Precision-Recall Area Under Curve
RF	Random Forest
ROC	Receiver Operating Characteristic
ROC-AUC	Receiver Operating Characteristic Area Under Curve
SVM	Support Vector Machine
XGB	Extreme Gradient Boosting

SYMBOLS AND NOTATIONS

α	Weighting factor
A_{pot}	Area potential score
$P(y=1 \mid X)$	Predicted probability of high-performing crop
RP	Revenue proxy (price $\times$ yield)
y_i	Target label for the i^{th} sample
Y_{pot}	Yield potential score

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

In all stages of the research presented in this paper, the usage of datasets from the Thanjavur district of the Tamil Nadu state, India, is used exclusively, and all analysis and evaluation are carried out with reference to this area only.

Until now, the selection of crops was based on experience, seasonality, and practice. These measures proved to be effective before but, in light of changing climatic conditions, decreasing soil fertility, and unpredictable agricultural markets, they are no longer efficient enough to achieve desired outcomes. In areas where farming plays an essential role, as in the case of Thanjavur district, the use of decision-making systems based on the data collected is crucial.

Thanks to developments in Artificial Intelligence (AI) and Machine Learning (ML), it is now possible to use heterogeneous data related to agriculture: information about the soil composition, climatic conditions, needs of certain crops, and their productivity in the past. The analysis of such data has evolved into the process of forecasting crop yields and providing relevant recommendations. Still, modern methods mostly rely either on agronomical factors alone or provide predictions without taking economics into account.

This study seeks to address the above-mentioned drawback through the utilization of the crop recommendation problem as a probability-oriented learning and ranking problem. Essentially, the crop recommendation model ranks crops based on the probability that such crops yield better performance in light of the present agricultural climatic conditions.

1.2 MOTIVATION

Motivation for the present research comes about due to the disparity between the existing information regarding agriculture and its actual application. There is a lot of fragmented data available to farmers and planning officers alike; however, what is lacking is a system that brings all this information together. This would ensure that crop

choice does consider the environment.

Technically speaking, modern ML algorithms used in agriculture are more likely to be black-box classifiers and do not provide any insight as to what drives their decision making process when generating predictions. At the same time, training an ML algorithm to perform tasks driven by financial motivations might result in choices being made in favor of profit, despite the unsuitability of the proposed solution.

The motivation behind this research is the development of a framework where prediction algorithm is applied to the target proxies that are strategically chosen along with features constraints. This can be achieved by ignoring the price as one of the features but keeping the target definition intact.

1.3 SCOPE OF THE PROJECT

The scope of this project shall be limited to the design of a ML-based framework for recommending crops that can be used in the Thanjavur district of the Indian state of Tamil Nadu. The project involves analyzing agricultural data and recommending crops based on the probability of success as well as other metrics.

From the functional perspective, the system implements data integration, preprocessing, feature extraction, and supervised learning of model functions with an aim to estimate the probability of crop performance classification. Additionally, it provides multiple rankings, including the probability ranking, yield ranking, and income ranking.

Aspects which are not functional in nature include interpretability, efficiency, and usability. The proposed model is meant to run on a computer without any particular specifications and can be used by anyone having ordinary computing facilities. Also, it becomes easier for people to understand the reasoning behind recommendations because of the clear-cut transparency in modeling and ranking processes.

From a technical perspective, the system can be used only for data modeling in tabular form using classic ML models. The system uses Python libraries for the processing of data and modeling, and cross-validation along with ranking-oriented performance measures like Precision-Recall Area Under Curve (PR-AUC) are used during the testing phase. The system does not have the capability to use streaming data, deep learning, or optimization techniques.

From a technical perspective, the system can be used only for data modeling in tabular form using classic ML models. The system uses Python libraries for the processing of data and modeling, and cross-validation along with ranking-oriented performance measures like PR-AUC are used during the testing phase. The system does not have the capability to use streaming data, deep learning, or optimization techniques.

The boundaries of the project will be dictated by assumptions and constraints. The first assumption made here concerns the datasets used in this work being accurate, representative of the region, and preprocessed. Moreover, the model works on the assumption that historical information about yields and prices can serve as a decent estimate of crop performance. In terms of constraints, the model uses fixed datasets, there is no temporal modeling, and the target is constructed based on proxy features instead of actual performance indicators.

CHAPTER 2

PROJECT DESCRIPTION AND GOALS

2.1 LITERATURE REVIEW

The application of ML in agriculture has gained significant attention over the past decade, driven by the increasing availability of structured agro-climatic datasets and advancements in computational methods. Several studies have demonstrated that supervised learning models such as Random Forest (RF) and Support Vector Machine (SVM) can achieve classification accuracies in the range of 80% to 92% for crop recommendation tasks when trained on soil and climatic features. Ensemble-based approaches, particularly Gradient Boosting (GB) and RF variants, have consistently outperformed single-model baselines due to their ability to capture nonlinear relationships and feature interactions.

More recent work has explored advanced boosting frameworks such as Extreme Gradient Boosting (XGB), Light Gradient Boosting Machine (LGBM), and CatBoost, which are capable of handling heterogeneous feature distributions and class imbalance more effectively. Empirical studies report that these models often achieve PR-AUC values above 0.80 in agricultural classification problems, making them suitable for ranking-based decision systems. However, these models are frequently optimized for predictive performance without sufficient consideration of interpretability or domain-specific constraints.

A parallel line of research has focused on integrating economic factors into agricultural decision-making. While some systems incorporate market price directly as an input feature, this approach introduces the risk of data leakage and may bias the model toward short-term economic signals. Other studies have proposed multi-objective frameworks that balance yield and profit, but these often require complex optimization techniques that reduce interpretability and practical usability.

Furthermore, evaluation methodologies in existing systems are often limited to accuracy and F1-score, which do not adequately capture the quality of ranked recommendations. In imbalanced datasets, where high-performing crop instances may represent less than 25% of the data, metrics such as PR-AUC and Receiver Operating Characteris-

tic Area Under Curve (ROC-AUC) provide a more reliable measure of model effectiveness. Despite these advancements, there remains a lack of integrated frameworks that simultaneously address agronomic suitability, economic relevance, and ranking performance.

Unlike many existing studies that focus on generalized or multi-region datasets, this work specifically targets the Thanjavur region of Tamil Nadu, enabling a more localized and context-aware analysis.

2.2 RESEARCH GAP

However, there are many areas that need to be considered even with the current advancements in ML-powered agricultural system solutions. For example, majority of the crop recommendation solutions developed till now treat this task as a simple classification problem, without taking into consideration the fact that it is a multi-class problem.

A further issue can be seen in terms of how the economic factors are handled. In systems where price serves as an input feature, high prediction accuracy is obtained but at the cost of generalization, since the system is dependent on any changes occurring in the market. On the other hand, systems which do not include any economic factor at all will be unable to predict profitability, which is essential to the success of the algorithm.

Furthermore, numerous studies fail to directly tackle issues related to class imbalance, whereby crops with high performance tend to reside in the highest quarter of the data set. The problem of class imbalance might cause bias towards the majority class if not handled properly. There is minimal focus on the ranking task, despite the fact that the ultimate aim is to rank the crops.

2.3 OBJECTIVES

The main objective in this research study is the creation of an ML-based approach whereby crop recommendation is perceived from the perspective of a probability ranking problem. The idea here is for the model to calculate the probability of making a suitable crop recommendation based on different attributes.

The next step in this process would involve designing an objective function that defines economic success without revealing any secrets through the feature vector. This

can be achieved by creating a surrogate measure for economic success based on a product of modal market price and historical yield of the top 25%. Note that the market price should not appear among the feature vectors.

The evaluation process aims to assess different models on increasingly richer feature spaces that consist exclusively of soil features, weather-soil features, or both feature spaces. Performance assessments are carried out using the stratified validation approach, where PR-AUC is used as the primary ranking accuracy metric.

Among other objectives, there is a need to develop a model ensemble for guaranteeing consistency and reliability. Furthermore, the objective includes the development of various crop rankings, which would be done with respect to probability, potential yield, and profits.

2.4 PROBLEM STATEMENT

In this research project, the research question being answered pertains to how to create a reliable model for determining the viability of planting different crops, with regard not only to its practicability but also the potential profits that might be generated through the procedure. Nonetheless, considering that no labels exist for the efficiency of each individual crop, some variables will have to be substituted.

From a technical perspective, this problem could be considered learning of a function that takes as its input an input feature space consisting of soil, weather, requirements, and productivity features and outputs the probability of being in a high-performing group for each of the crops. This way, crops are ranked based on their probabilities.

2.5 PROJECT PLAN

Regarding the implementation of the proposed framework to ensure the achievement of the outlined goal, there will be carried out a series of phases, which are closely connected with each other. The first phase will include the formulation of the problem itself, together with its goals and the needed literature review that would help establish the theoretical basis of the project. Following that, agricultural data collection will be performed with particular interest put on pre-processing, normalization, and regional

matching concerning the Thanjavur district. Then, there will occur the phase of feature engineering and target creation, where, respectively, predictive features will be created, and the target feature will be established without leakage. Also, different types of machine learning models have to be considered, meaning comparing them based on their efficiency and using stratification and ranking measures as the criteria of evaluation. Therefore, an ensemble model will be designed, and after that, the final step, involving calculation of probabilities, productivities, and profitabilities, as well as crop ranking, will take place.

CHAPTER 3

TECHNICAL SPECIFICATION

3.1 SYSTEM OVERVIEW

The designed system is realized as a modular ML pipeline that is specific for the Thanjavur area in the state of Tamil Nadu and uses agricultural datasets from the said region to make crop recommendations based on that area. The system can be extended by including more sources of data and applying new models.

The system processes structured data tables through the use of the Python programming language with the help of popular data processing libraries like Pandas and NumPy, while machine learning models are created using the scikit-learn library. There are also some optional models that include XGB, LGBM, and CatBoost.

3.2 DATA SPECIFICATION

In addition to that, there are different kinds of datasets depending on the origin in the feature domain. For instance, the soil dataset will have the composition of various elements such as nitrogen, phosphorus, and potassium. On the other hand, weather dataset will contain average figures of temperatures, rainfall and humidity. Besides, crop needs dataset will have optimum environmental conditions, while the agriculture produce dataset will have average land production.

The combined dataset contains both continuous and categorical features, requiring careful preprocessing and normalization. Typical feature dimensions range from 15 to 40 attributes depending on the selected feature set, with sample sizes varying based on crop coverage and data availability collected from various Application Programming Interface (API).

All datasets used in this study are filtered and aligned to represent agricultural conditions within the Thanjavur district.

3.3 FEATURE ENGINEERING

Feature engineering is another vital stage that will contribute to the unification of data into a single model. The crops' names will be standardized so that the process of mapping will be consistent. Normalization or standardization may be used as needed with regards to the numeric features. Derived features will be developed in order to identify any significant relationships such as the ratio of yields to areas.

Several different feature sets are taken into account to assess the influence of different domains on the analysis process. The first basic feature set includes only two domains—soil and weather, whereas the remaining extended feature sets include some additional information related to the crops' requirements domain, as well as past information about production. From empirical observation, it is seen that more advanced feature sets work much better than the basic ones for PR-AUC.

3.4 TARGET CONSTRUCTION

Due to the lack of direct observations about the crop's performance, the creation of a surrogate target variable becomes inevitable. The revenue-based approach of defining the surrogate target is the multiplication of the modal market value and previous year's yield of the crop. The data set is split into two sets according to the revenue-based surrogate target, where those in the top quartile (greater than equal to 75th percentile) are considered high-performers and the others, low-performers.

This method guarantees that roughly 25% of the dataset contains the positive class, thereby creating a deliberate imbalance in the data set. The critical point to note is that market prices are not considered when selecting input features to avoid any possible data leakage issues.

3.5 MODELING FRAMEWORK

The algorithm considers several classes of models in order to account for different types of learning behavior. Ensemble tree models like RF and GB are used due to their capacity to represent non-linear relationships and deal with heterogeneities among features. Logistic Regression (LR) is considered the linear baseline, whereas SVM with radial basis functions offers a margin-based classifier method.

If applicable, the more sophisticated boosting models like XGB, LGBM, and CatBoost are employed. The use of such models is highly effective for working with structured data and imbalanced classification problems, with very good rank predictions often observed. Empirical evidence suggests that ensemble and boosting models produce almost identical PR-AUC scores.

3.6 TRAINING AND VALIDATION STRATEGY

Splitting is done in an 80:20 ratio between training and test datasets, with the classes being preserved due to stratification. In order to make sure that the results produced are more stable, a 5 fold Stratified K Fold Cross Validation (CV) technique is used on the training set.

Class imbalance is solved using class balancing in the respective models along with scaling in boosting algorithms. Accuracy, F1-score, ROC-AUC, and PR-AUC are some of the evaluation metrics used. Among these, PR-AUC is considered the primary metric because of its correlation with class imbalance and ranking problems.

Model selection is done according to a hierarchical principle, where the top priority goes to PR-AUC, followed by F1-score and accuracy, guaranteeing that the selected models are ranked well but not necessarily classified accurately.

3.7 SYSTEM IMPLEMENTATION

The implementation consists of a reproducible pipeline of data loading, preprocessing, feature engineering, model training, and testing inside the notebook. In order to help understand and compare various models, outputs such as leaderboard and visualization of results will be produced in between steps.

The architecture allows for graceful degradation with respect to optional modules. It will still run when not all models can be built due to lack of relevant libraries, and only use available models in that case.

3.8 HARDWARE AND SOFTWARE ENVIRONMENT

It was created and tested on a computer that contained an Intel Core i5 (13th gen) processor, 16 gigabytes of RAM, and NVIDIA GeForce RTX 2050 GPU. This configu-

ration had enough power to calculate all the ML processes from preprocessing, feature extraction, cross-validation, and ensemble creation.

The use of the multi-core processing unit allowed performing computations in parallel mode, which was necessary in the case of calculating decision trees and their ensembles, such as RF and GB. The presence of RAM in an appropriate amount enabled processing moderate-sized tabular datasets and the features extracted from them without any issues with limited resources. Even though currently implemented modeling tools utilize traditional machine learning models that can be effectively calculated on CPU, the presence of CUDA-compatible Graphics Processing Unit (GPU) allows considering future advancements in GPU-based boosting and neural network algorithms.

The development environment uses Python (version 3.8 or above) as the programming language, and the major packages that will be utilized include Pandas, NumPy (for data handling), scikit-learn (for model building and validation), and Matplotlib (for visualization). Optional modules like XGB, LGBM, and CatBoost can also be included to increase variety in terms of models. All code implementation is carried out in the form of a notebook environment.

CHAPTER 4

SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

The proposed framework will be developed as a ML architecture with a hierarchical organization of heterogeneous agricultural data sources, leading to probability-based ranking of crops. It will be built upon a data-centric architecture where several different types of feature domains are carefully integrated into one data model to enable effective analysis.

In terms of data representation, the proposed system employs a single feature set, which consists of soil properties, weather data, crop requirements, and historical productivity indicators. The mentioned sources of data are preprocessed and aligned to create a single feature space, which is further used during the analysis.

Overall, the proposed architecture will consist of three main layers. Data ingestion and preprocessing will comprise the first layer, where datasets will be properly cleaned and normalized. The next layer focuses on feature engineering and classifier training, where classifiers are trained based on progressively enriched feature space. The third layer of our system represents an implementation of ensemble aggregation and ranking of crops.

This layered design ensures modularity and allows individual components, such as feature engineering or model selection, to be modified independently without affecting the overall system.

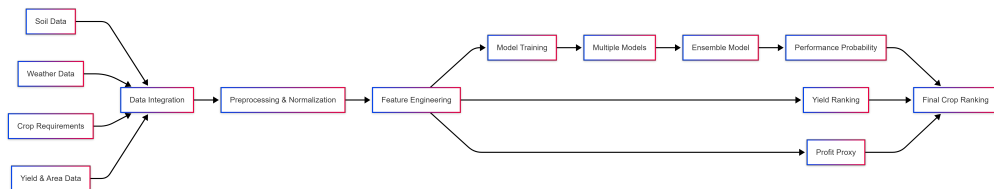


Figure 4.1: System Architecture

4.2 DATA FLOW AND PROCESSING PIPELINE

The proposed system uses a sequential pipeline for processing the agriculture data from the raw data to recommendation generation process. First, a data loading step is considered that involves the loading of multiple CSV file format datasets where region-wise records are retained. Preprocessing steps are taken into consideration to address any inconsistencies in the datasets such as normalization of the crop names as well as converting numerical data into the required format.

After pre-processing, the data undergoes feature engineering to generate relevant features. Productivity measures like area/yield ratio generation and integration of crop requirement compatibility are carried out during the process. These features are further divided into multiple feature sets to allow the comparison of different models' performance according to their information availability.

Partitioning the dataset into a training and testing set is done through a stratified split technique to maintain class balance.

4.3 MODEL DESIGN

The modeling module of the proposed system aims at evaluating various classifier families, which would provide insight into various properties of the data distribution. Models like tree-based ensembles (RF, GB) help capture non-linear relationships and feature interactions, while LR, SVM etc., help understand decision boundaries in linear space.

Various models are trained based on different feature sets in order to check the importance of the new set of features. It has been found that the feature set with all the possible attributes, i.e. soil, weather, requirement, and productivity feature sets give the best results. It can be seen from experience that this particular feature set gives consistent performance improvements for ranking measures like PR-AUC.

Stratified cross-validation techniques are used for training the models and evaluation is done using a combination of measures which not only evaluate classification but also measure ranking power of the models.

4.4 ENSEMBLE DESIGN AND RANKING MECHANISM

This approach makes use of an ensemble approach in which the forecasts that are being generated by the best models belonging to each model family are combined. The best forecasts in each category are referred to as winning forecasts, while the ensemble forecast is produced based on the probability forecasts of crop suitability generated by the winners.

The suggestions are made based on the rankings performed independently in three categories. In the first ranking, probabilities are used to compute probabilities according to the chance that the crop would have high suitability. In the second ranking, past year statistics on productivity will be used. The third ranking will use a revenue-based measure as the metric.

4.5 DESIGN CONSIDERATIONS

There are several factors that can have an effect on how the system is designed. First, the prevention of the leakage of the features is done by excluding the market price from the input features and including it into the target definition. Second, proxy labeling is used as a factor which allows building the system not using ground truth labels. Finally, the importance of ranking is considered as a way to fit the task with the objectives.

Besides, the modularity of the system can be utilized to introduce new datasets, features, and models easily.

CHAPTER 5

METHODOLOGY AND TESTING

5.1 FEATURE ENGINEERING

5.1.1 Normalized Productivity Features

$$\text{Normalized Yield Score} = \frac{Y_i - Y_{\min}}{Y_{\max} - Y_{\min}} \quad (5.1)$$

where y_i represents the median yield of crop i .

$$\text{Area Score} = \frac{A_i - A_{\min}}{A_{\max} - A_{\min}} \quad (5.2)$$

where A_i represents cultivated area.

5.1.2 Yield and Area Potential

$$Y_{\text{pot}} = 100 \times \frac{\text{rank}(Y_i)}{N} \quad (5.3)$$

$$A_{\text{pot}} = 100 \times \frac{\text{rank}(A_i)}{N} \quad (5.4)$$

where Y_{pot} and A_{pot} represent yield and area potential, respectively.

5.1.3 Combined Productivity Score

$$\text{Yield Combined Score} = 0.7 \times Y_{\text{pot}} + 0.3 \times A_{\text{pot}} \quad (5.5)$$

5.2 TARGET CONSTRUCTION

$$RP_i = \text{Modal Price} \times \text{Historical Yield} \quad (5.6)$$

where RP represents the revenue proxy.

$$y_i = \begin{cases} 1, & \text{if } RP_i \geq Q_{0.75} \\ 0, & \text{otherwise} \end{cases} \quad (5.7)$$

5.3 MODEL TRAINING

$$P(y = 1 | X) = f(X; \theta) \quad (5.8)$$

where $P(y = 1 | X)$ represents the predicted probability.

$$\hat{y} = \arg \max_{y \in \{0,1\}} P(y | X) \quad (5.9)$$

5.4 VALIDATION AND EVALUATION

$$\text{Precision} = \frac{TP}{TP + FP} \quad (5.10)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (5.11)$$

5.5 ENSEMBLE CONSTRUCTION

$$P_{\text{ensemble}} = \frac{1}{M} \sum_{i=1}^M P_i \quad (5.12)$$

5.6 RANKING MECHANISM

$$\text{Final Score} = w_1 P_{\text{ensemble}} + w_2 Y_{\text{score}} + w_3 R_{\text{score}} \quad (5.13)$$

where w_1 , w_2 , and w_3 represent weighting parameters.

CHAPTER 6

PROJECT IMPLEMENTATION

6.1 DATA HANDLING AND INTEGRATION

The implementation starts by introducing multiple data sets, which include soil characteristics, weather summaries, crop needs, and past productivity. Data sets are kept in Comma Separated Values (CSV) format and read using data processing modules. The data sets are filtered based on region to ensure that the Thanjavur data set is considered.

Data consolidation is carried out using the mapping of crops in different data sets. Crop names are standardized in the preprocessing stage, and any missing data is handled. Numerical data attributes are converted into their correct forms during the process.

6.2 FEATURE ENGINEERING IMPLEMENTATION

Feature engineering is done using both direct transformation as well as derived transformations. Nutrient values and environmental variables are left as raw data whereas new features are defined for improving the expressive capability of the model.

Ratio of yield per area value is calculated to bring normalization in productivity for different crops with different sizes of cultivation areas. Compatibilities are another set of features that are calculated from observations compared to the required ranges for different crops.

More than one set of features is generated to enable comparative experiments. Different sets of features can be selected during execution to evaluate the effect of data domains on models.

6.3 MODEL IMPLEMENTATION

The modeling phase is performed using common machine learning packages. Several classes of classifiers are initialized and trained using the preprocessed dataset. Each model is provided with optimized hyperparameters acquired using cross-validation.

Ensemble algorithms like RF and GB are introduced to account for any nonlinearity present. Models that rely on linearity like LR and SVM are used as benchmarks.

Boosting libraries like XGB and LGBM are incorporated if possible.

Table 6.1: Summary of Models and Key Hyperparameters

Model	Key Hyperparameters
Random Forest	n_estimators=300–500, max_depth=10–None, class_weight=balanced
Gradient Boosting	n_estimators=100–300, learning_rate=0.03–0.1, max_depth=3–5
Logistic Regression	C=0.01–10, penalty=L1/L2, solver=liblinear/saga
Support Vector Machine	C=0.1–10, kernel=RBF, gamma=scale/auto
XGBoost	n_estimators=200–500, learning_rate=0.03–0.1, max_depth=4–8
LightGBM	n_estimators=200–500, num_leaves=31–127, learning_rate=0.03–0.1
CatBoost	iterations=200–500, depth=4–8, learning_rate=0.03–0.1

Training is performed using a stratified data split, ensuring that class distribution is preserved. The implementation includes mechanisms for handling class imbalance through weighting strategies and parameter adjustments.

6.4 VALIDATION AND EVALUATION IMPLEMENTATION

Model evaluation is implemented using a StratifiedKFold-based cross-validation framework. This approach ensures that each fold maintains the class distribution, providing reliable performance estimates.

Evaluation metrics include accuracy, F1-score, ROC-AUC, and PR-AUC. The implementation emphasizes PR-AUC as the primary metric due to its relevance in ranking scenarios with imbalanced data. Performance results are aggregated across folds to obtain stable estimates.

Intermediate outputs, including performance tables and metric summaries, are generated to facilitate model comparison and selection.

6.5 ENSEMBLE AND RANKING IMPLEMENTATION

The ensemble mechanism is implemented by aggregating predicted probabilities from the best-performing models. For each selected model, probability estimates are computed on the validation and test sets. These probabilities are then averaged to produce a final ensemble score.

The ranking system is constructed using three parallel computations. The first ranking is based on ensemble probabilities, representing predicted performance. The second

ranking is derived from historical productivity metrics, while the third ranking is based on the revenue proxy.

The implementation ensures that these rankings are generated independently and then presented together, allowing users to compare different aspects of crop performance.

6.6 SYSTEM WORKFLOW EXECUTION

The complete system operates as a sequential pipeline. Data is loaded and preprocessed, followed by feature construction and dataset preparation. Models are then trained and evaluated using CV, after which the ensemble is constructed. Finally, rankings are generated and presented as output.

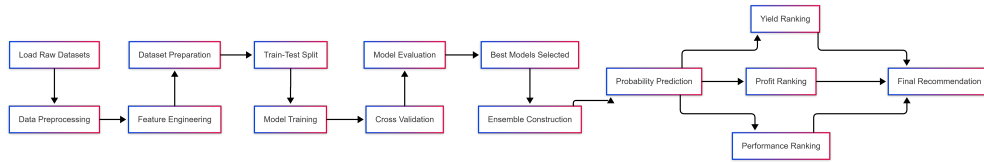


Figure 6.1: Workflow Diagram

The workflow is designed to be reproducible, with each stage clearly defined and executed in sequence. The use of a notebook-based implementation allows intermediate results to be inspected and validated, ensuring transparency in the modeling process.

6.7 IMPLEMENTATION CHALLENGES

During the process of implementing the solution to the problem, a number of challenges came up. The first challenge was dealing with inconsistencies in the crops' names. To solve this, crop normalization had to be done. Since the dataset involved some class imbalance, an appropriate weighting approach had to be employed.

The following were the means by which this was achieved.

CHAPTER 7

RESULTS AND DISCUSSION

7.1 MODEL PERFORMANCE ANALYSIS

Table 7.1: Performance Comparison of Models on Holdout Set (Best Feature Set)

Model	PR-AUC	F1 Score	Accuracy	ROC-AUC
Random Forest	0.8323	0.7301	0.8251	0.9442
XGBoost	0.8323	0.7301	0.8251	0.9442
Gradient Boosting	0.8323	0.7007	0.8824	0.9442
CatBoost	0.8323	0.7301	0.8251	0.9442
LightGBM	0.8323	0.7301	0.8251	0.9442
Logistic Regression	0.8169	0.6930	0.8067	0.9341
SVM	0.4829	0.7301	0.8251	0.8244

It can be seen from Table 7.1 that most models perform almost identically, with all their PR-AUC scores falling around 0.83. There is an indication of better performance from tree-based and boosting models such as RF, GB, XGB, and LGBM; however, the difference between different models' performance is not very significant.

This low variation in performance implies that the predictive power of the system mainly comes from features and data representation, and not from the complexity of the model itself. This further means that linear models like LR perform similarly to other models. The low PR-AUC value of SVM suggests poor performance at ranking.

7.2 FEATURE CONTRIBUTION ANALYSIS

Table 7.2: Best Model Performance Across Feature Sets

Feature Set	Model	PR-AUC	F1	Accuracy	ROC-AUC
Soil + Weather + Requirements + Yield	Random Forest	0.8323	0.7301	0.8251	0.9442
Soil + Weather + Requirements	Random Forest	0.8058	0.6820	0.7692	0.9318

With the incorporation of productivity-based parameters, there is definitely a huge improvement in performance. The introduction of a larger number of parameters means that the performance is higher than the default scenario in terms of PR-AUC, which

clearly emphasizes the importance of incorporating historical productivity information and data on land.

Weather conditions become vital for prediction, while soil nutrition makes for suitable inputs.

7.3 EFFECT OF HYPERPARAMETER TUNING

Hyperparameter tuning contributes to performance stability rather than large performance gains. Models with moderate complexity, such as RF with controlled depth and boosting models with learning rates between 0.03 and 0.1, demonstrate consistent results across validation folds.

Excessively complex configurations lead to overfitting, while overly simple models fail to capture important feature interactions. These observations validate the chosen hyperparameter ranges and highlight the importance of balanced model design.

7.4 ENSEMBLE PERFORMANCE

The ensemble approach aggregates predictions from multiple models to improve robustness. While individual models already achieve similar PR-AUC values, the ensemble provides improved stability in predictions rather than a significant increase in absolute performance.

This behavior indicates that ensemble learning helps reduce variance and ensures consistent ranking across different data splits, making it more reliable for decision-making.

7.5 RANKING BEHAVIOR AND INTERPRETATION

The system generates three complementary rankings based on ensemble probability, yield potential, and revenue proxy. The probability-based ranking reflects predicted suitability, while the yield-based ranking emphasizes productivity. The revenue-based ranking incorporates economic considerations.

The consistency observed across these rankings for top crops indicates that the system effectively captures both agronomic and economic signals. However, differences between rankings highlight the trade-offs between productivity and profitability.

7.6 FINAL CROP RANKING AND RECOMMENDATION

The ranking outputs are representative of crop suitability within the Thanjavur region and should be interpreted within this localized agricultural context.

Table 7.3: Top Crops by Predicted High-Performance Probability

Crop	Mean Probability
Banana	0.9635
Tapioca	0.9606
Garlic	0.9600
Coconut	0.6607
Lemon	0.6446

Table 7.4: Top Crops by Yield Potential

Crop	Yield Score
Coconut	93.70
Banana	92.3
Tapioca	90.6
Maize	76.9
Guava	75.2

Table 7.5: Top Crops by Profit Proxy

Crop	Profit Score
Banana	86.2
Tapioca	82.8
Garlic	79.3
Lemon	75.9
Onion	72.4

These results demonstrate strong agreement across ranking strategies, particularly for crops such as Banana and Tapioca, which consistently appear among the top recommendations. This consistency reinforces the reliability of the proposed framework.

It is important to note that these rankings are derived from raw soil and environmental data and do not fully account for real-world agricultural interventions such as fertilizer application, irrigation, and pest management. As a result, the recommendations reflect inherent land suitability rather than fully optimized farming conditions.

7.7 DISCUSSION

The results demonstrate that integrating multiple data domains with ML techniques produces stable and interpretable crop recommendations. The probabilistic formulation enables flexible ranking, improving usability compared to rigid classification approaches.

The use of proxy-based target construction successfully incorporates economic considerations without introducing feature leakage. The emphasis on PR-AUC ensures that evaluation aligns with ranking objectives under class imbalance.

7.8 LIMITATIONS

A key limitation arises from the absence of granular fertilizer usage data. The Thanjavur dataset reflects relatively lower nitrogen levels, leading to implicit penalization of nitrogen-intensive crops.

In real-world conditions, fertilizer application significantly alters soil nutrient availability, enabling such crops to perform effectively. The lack of this information introduces a gap between predicted suitability and actual performance.

This limitation highlights a data constraint rather than a modeling issue. Incorporating fertilizer application data would improve prediction accuracy and reduce bias against nutrient-dependent crops.

CHAPTER 8

CONCLUSION AND FUTURE WORK

8.1 CONCLUSION

This paper introduces a region-specific ML-powered crop recommendation model for the Thanjavur district in Tamil Nadu state. In order to make recommendations, the model combines information about the physical parameters of the land, climate conditions, crop preferences, and production history, which enables it to rank crops with respect to their potential for successful growth within the specified agro-climate.

The experimental analysis revealed that the suggested method provides stable and consistent results across different models. The tree-based models and boosting algorithms, such as RF, GB, XGB, and LGBM, exhibit similar results; their PR-AUC scores lie in the vicinity of 0.83 on the holdout sample set. Thus, it can be said that the prediction power of the model is due to adequate feature engineering rather than the complexity of the model.

Ensemble learning is another way to improve the robustness of the predictions, with results from several algorithms combined. Furthermore, the application of the multi-dimensional ranking approach based on probability, potential yield, and profit allows us to achieve a balanced agronomic and economically oriented decision-making process.

The model successfully shows that using a combination of various agriculture data sources along with ML approaches enables generating meaningful recommendations. The results need to be considered as an initial estimate for the suitability of crops in the Thanjavur area, given that they were obtained using the primary data related to soil and environmental factors.

8.2 FUTURE WORK

Although the suggested model offers an excellent base for building crop recommendations using data, there are still some improvements that can be considered to enhance its efficiency and effectiveness. The first area where improvements can be made is integrating information about the usage of fertilizers in terms of their amounts and frequency. It would help eliminate the shortcomings associated with nutrient availability

in nitrogen-loving crops and minimize deviations from the model predictions.

Another improvement that can be taken into consideration is taking time and seasonal changes into account. Introducing weather forecasts and seasonal changes would allow making more accurate and timely recommendations based on the current situation.

Furthermore, the system can be enhanced to incorporate other agronomic aspects such as the use of irrigation, occurrence of pests/diseases, as well as soil water movement. Such parameters will have an effect on the behavior of the crops, thus making the recommendation more realistic.

Regarding future development from a modeling standpoint, the application of other ML methods and hybrid systems to model nonlinearities in the system is one possible area for improvement. Moreover, deploying the entire system as either a web or mobile application could provide farmers the opportunity to get recommendations in real-time.

In conclusion, the proposed framework provides an extensible framework for developing agricultural decision support systems, which can still benefit from improvement by considering other features, as discussed above.

BIBLIOGRAPHY

- A, P., Chakraborty, S., Kumar, A., and Pooniwala, O. R. (2021). Intelligent crop recommendation system using machine learning. In *2021 5th International Conference on Computing Methodologies and Communication (ICCMC)*, pages 843–848.
- Agarwal, A., Ahmad, S., and Pandey, A. (2023). Crop recommendation based on soil properties: A comprehensive analysis. In *2023 14th International Conference on Computing Communication and Networking Technologies (ICCCNT)*, pages 1–6.
- Bag, S., Padhy, N., and Panigrahi, R. (2025). Enhancing crop recommendation systems using deep learning techniques on soil and environmental data. In *2025 International Conference on Emerging Systems and Intelligent Computing (ESIC)*, pages 601–606.
- Department, T. N. A. (2026). Tamil mannvalam.
- Dolli, Rawat, P., Bajaj, M., Vats, S., and Sharma, V. (2023). An analysis of crop recommendation systems employing diverse machine learning methodologies. In *2023 International Conference on Device Intelligence, Computing and Communication Technologies, (DICCT)*, pages 619–624.
- Doshi, Z., Nadkarni, S., Agrawal, R., and Shah, N. (2018). Agroconsultant: Intelligent crop recommendation system using machine learning algorithms. In *2018 Fourth International Conference on Computing Communication Control and Automation (IC-CUBE)*, pages 1–6.
- Jaichandran, R., Krishna, T. M., Arigela, S. H., Raman, R., Dharani, N., and Kumar, A. (2022). Light gbm algorithm based crop recommendation by weather detection and acquired soil nutrients. In *2022 International Conference on Power, Energy, Control and Transmission Systems (ICPECTS)*, pages 1–5.
- Kulkarni, N. H., Srinivasan, G. N., Sagar, B. M., and Cauvery, N. K. (2018). Improving crop productivity through a crop recommendation system using ensembling technique. In *2018 3rd International Conference on Computational Systems and Information Technology for Sustainable Solutions (CSITSS)*, pages 114–119.

- Modi, D., Sutagundar, A. V., Yalavigi, V., and Aravatagimath, A. (2021). Crop recommendation using machine learning algorithm. In *2021 5th International Conference on Information Systems and Computer Networks (ISCON)*, pages 1–5.
- Motwani, A., Patil, P., Nagaria, V., Verma, S., and Ghane, S. (2022). Soil analysis and crop recommendation using machine learning. In *2022 International Conference for Advancement in Technology (ICONAT)*, pages 1–7.
- Open-Meteo (2026). Open-meteo.
- Pudumalar, S., Ramanujam, E., Rajashree, R. H., Kavya, C., Kiruthika, T., and Nisha, J. (2017). Crop recommendation system for precision agriculture. In *2016 Eighth International Conference on Advanced Computing (ICoAC)*, pages 32–36.
- S, P. and K, V. (2025). Optimizing crop yield - an iot and ml-based soil testing and crop recommendation system. In *2025 3rd International Conference on Intelligent Data Communication Technologies and Internet of Things (IDCIoT)*, pages 789–795.
- Sudhakaran, P., Gnanavel, V. K., Gunasekaran, K., S, A., and M, P. (2025). Ai-powered leaf disease detection and crop recommendation system. In *2025 2nd International Conference on Computing and Data Science (ICCDs)*, pages 1–5.
- Tomar, V., Sharma, G., Rajkumar, R., Singh, A., Dhakare, S. S., and Kumar, K. (2024). Development of a machine learning-based system for optimizing crop recommendations. In *2024 1st International Conference on Advances in Computing, Communication and Networking (ICAC2N)*, pages 1594–1599.
- University, T. N. A. (2026). Tnau agriportal.

Appendix A- Source Code

Thanjavur Crop Recommendation - Clean ML Pipeline

This notebook contains only the production flow:

1. Imports
2. Exploratory Data Analysis (EDA)
3. Data preprocessing
4. Dataset creation
5. Train/test split
6. Model training
7. Results
8. Crop prediction

1. Imports

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
from pathlib import Path
import glob

from sklearn.base import clone
from sklearn.model_selection import train_test_split, StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier,
    GradientBoostingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, f1_score,
    average_precision_score, roc_auc_score, classification_report

# Optional advanced boosting libraries

try:
```



```

    from catboost import CatBoostClassifier
except ImportError:
    CatBoostClassifier = None

try:
    from xgboost import XGBClassifier
except ImportError:
    XGBClassifier = None

try:
    from lightgbm import LGBMClassifier
except ImportError:
    LGBMClassifier = None

warnings.filterwarnings('ignore')
sns.set_style('whitegrid')
plt.rcParams['figure.figsize'] = (12, 6)

data_path = Path('Data')
print(f'Data path: {data_path}')
## 2. Exploratory Data Analysis (EDA)

This section performs quick data-quality and distribution checks before
    feature engineering:

- Dataset availability and shape
- Missing-value diagnostics
- Key numeric distribution snapshots
- Soil nutrient and weather variability checks
# EDA: Load core datasets for quality checks
soil_file_eda = data_path / 'Soil Data ( District Wise)' / 'CSV Format'
    / 'THANJAVUR.csv'
weather_file_eda = data_path / 'Weather Data (District Wise)' / '
    weather_data_all_blocks.csv'

```

```

requirements_file_eda = data_path / 'crop_requirements.csv'
area_yield_file_eda = data_path / 'Crop Area And Yield Data.csv'

soil_eda = pd.read_csv(soil_file_eda)
weather_eda = pd.read_csv(weather_file_eda)
requirements_eda = pd.read_csv(requirements_file_eda)
area_yield_eda = pd.read_csv(area_yield_file_eda)

crop_files_eda = glob.glob(str(data_path / '3_Cleaned CSVs' / '*.csv'))

print('EDA Dataset Overview')
print('=' * 80)
print(f'Soil (Thanjavur): {soil_eda.shape}')
print(f'Weather (all blocks): {weather_eda.shape}')
print(f'Crop requirements: {requirements_eda.shape}')
print(f'Area/Yield history: {area_yield_eda.shape}')
print(f'Crop market files found: {len(crop_files_eda)}')

if crop_files_eda:
    sample_market = pd.read_csv(crop_files_eda[0])
    print(f'Sample market file: {Path(crop_files_eda[0]).name} -> {
        sample_market.shape}')

print('\nMissing Values (Top columns)')
print('-' * 80)
for name, df in [
    ('soil_eda', soil_eda),
    ('weather_eda', weather_eda),
    ('requirements_eda', requirements_eda),
    ('area_yield_eda', area_yield_eda),
]:
    na = df.isna().sum().sort_values(ascending=False)
    na = na[na > 0].head(10)
    print(f'\n{name}:')

```

```

    if len(na) == 0:
        print(' No missing values')
    else:
        print(na.to_string())
# EDA: Quick visual diagnostics
plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
soil_numeric_cols = ['n_High', 'n_Medium', 'n_Low', 'p_High', 'p_Medium',
                    , 'p_Low', 'k_High', 'k_Medium', 'k_Low']
soil_profile = soil_eda[soil_numeric_cols].mean().sort_values(ascending=
    False)
sns.barplot(x=soil_profile.values, y=soil_profile.index, palette='
    viridis')
plt.title('Average Soil Nutrient Status (Thanjavur)')
plt.xlabel('Mean Value')

plt.subplot(1, 3, 2)
weather_subset = weather_eda[weather_eda['district'].astype(str).str.
    lower() == 'thanjavur'].copy()
if len(weather_subset) > 0 and 'total_rainfall_mm' in weather_subset.
    columns:
    sns.histplot(weather_subset['total_rainfall_mm'].dropna(), bins=20,
        kde=True, color='#2E8B57')
    plt.title('Rainfall Distribution (Thanjavur Blocks)')
    plt.xlabel('Total Rainfall (mm)')
else:
    plt.text(0.5, 0.5, 'No Thanjavur rainfall data', ha='center', va='
        center')
    plt.title('Rainfall Distribution')

plt.subplot(1, 3, 3)
if crop_files_eda:
    market_df_eda = pd.read_csv(crop_files_eda[0])

```

```

modal_col = 'Modal Price (Rs./Quintal)'
if modal_col in market_df_eda.columns:
    prices = pd.to_numeric(market_df_eda[modal_col], errors='coerce'
).dropna()
    if len(prices) > 0:
        sns.boxplot(y=prices, color='#4682B4')
        plt.title(f'Modal Price Spread\n{Path(crop_files_eda[0]).
stem}')
        plt.ylabel('Rs./Quintal')
    else:
        plt.text(0.5, 0.5, 'No numeric modal prices', ha='center',
va='center')
        plt.title('Market Price Spread')
    else:
        plt.text(0.5, 0.5, 'Modal price column not found', ha='center',
va='center')
        plt.title('Market Price Spread')
else:
    plt.text(0.5, 0.5, 'No crop market files found', ha='center', va='
center')
    plt.title('Market Price Spread')

plt.tight_layout()
plt.show()

print('EDA quick checks complete. Proceed to preprocessing and feature
    engineering.')
## 3. Data Preprocessing
# Load Thanjavur soil data
soil_file = data_path / 'Soil Data ( District Wise)' / 'CSV Format' / '
    THANJAVUR.csv'
soil_data = pd.read_csv(soil_file)

# Load weather data

```

```

weather_file = data_path / 'Weather Data (District Wise)' / '
    weather_data_all_blocks.csv'
weather_data = pd.read_csv(weather_file)
thanjavur_weather = weather_data[weather_data['district'] == 'Thanjavur'
    ].copy()

# Helper for consistent joins across market, requirements, and area/
yield files
def normalize_crop_name(name):
    if pd.isna(name):
        return ''
    text = str(name).strip().lower()
    replacements = {
        '-': '',
        '_': '',
        ' ': '',
        '(': '',
        ')': '',
        '/': '',
        '.': ''
    }
    for old, new in replacements.items():
        text = text.replace(old, new)
    return text

# Load all crop CSVs (market data)
crop_files = glob.glob(str(data_path / '3_Cleaned CSVs' / '*.csv'))
crop_data_dict = {}
for file in crop_files:
    crop_name = Path(file).stem
    try:
        df = pd.read_csv(file)
        crop_data_dict[crop_name] = df
    except:

```

```

pass

# Load crop requirements
requirements_file = data_path / 'crop_requirements.csv'
crop_requirements_df = pd.read_csv(requirements_file)
crop_requirements_df['crop_key'] = crop_requirements_df['Crop'].apply(
    normalize_crop_name)

req_level_map = {'low': 1, 'medium': 2, 'high': 3}
for col in ['N_Req', 'P_Req', 'K_Req']:
    crop_requirements_df[f'{col}_level'] = crop_requirements_df[col].
        astype(str).str.strip().str.lower().map(req_level_map).fillna(2)

# Load crop area/yield history and aggregate by crop
area_yield_file = data_path / 'Crop Area And Yield Data.csv'
area_yield_df = pd.read_csv(area_yield_file)
area_yield_df['crop_key'] = area_yield_df['Crop'].apply(
    normalize_crop_name)
area_yield_df['Area Under'] = pd.to_numeric(area_yield_df['Area Under'],
    errors='coerce')
area_yield_df['Yield'] = pd.to_numeric(area_yield_df['Yield'], errors='
    coerce')

crop_area_yield_agg = (
    area_yield_df.groupby('crop_key')[['Area Under', 'Yield']]
        .median()
        .rename(columns={'Area Under': 'area_median', 'Yield': 'yield_median'
            })
        .reset_index()
)

crop_area_yield_agg['yield_per_area'] = (
    crop_area_yield_agg['yield_median'] / crop_area_yield_agg['
        area_median'].replace(0, np.nan)
).replace([np.inf, -np.inf], np.nan)

```

```

print(f"Soil data shape: {soil_data.shape}")
print(f"Weather data shape: {thanjavur_weather.shape}")
print(f"Crops loaded: {len(crop_data_dict)}")
print(f"Crop requirements loaded: {crop_requirements_df.shape[0]}")
print(f"Crop area/yield rows loaded: {area_yield_df.shape[0]}")

## 4. Dataset Creation

# Create soil features
soil_summary = soil_data.groupby('District')[[
    'n_High', 'n_Medium', 'n_Low',
    'p_High', 'p_Medium', 'p_Low',
    'k_High', 'k_Medium', 'k_Low',
    'pH_Neutral', 'pH_Acidic', 'pH_Alkaline',
    'EC_Saline', 'EC_NonSaline',
    'OC_High', 'OC_Medium', 'OC_Low'
]].mean()

# Create weather features
weather_features = thanjavur_weather[[
    'temp_max_mean', 'temp_min_mean', 'temp_mean_annual',
    'total_rainfall_mm', 'avg_daily_rainfall_mm',
    'humidity_max_mean', 'humidity_min_mean',
    'rainy_days', 'wind_speed_max_mean'
]].mean()

print("Soil Features:")
print(soil_summary.loc['THANJAVUR'])
print("\nWeather Features:")
print(weather_features)

## 5. Train/Test Split

# Create a dataset where EACH TRANSACTION is a separate row

# - Features: soil + weather + crop requirements + historical area/yield
      (EXCLUDING price for suitability)

# - Target: HIGH-PERFORMING transaction (1) vs lower-performing (0)

```

```

tf_soil = soil_summary.loc['THANJAVUR']

# Build lookup tables for crop-specific features
requirements_lookup = crop_requirements_df.set_index('crop_key')[
    ['N_Req_level', 'P_Req_level', 'K_Req_level', 'Rainfall', 'Temp']
]
area_yield_lookup = crop_area_yield_agg.set_index('crop_key')[
    ['area_median', 'yield_median', 'yield_per_area']
]

default_req = pd.Series({
    'N_Req_level': 2.0,
    'P_Req_level': 2.0,
    'K_Req_level': 2.0,
    'Rainfall': crop_requirements_df['Rainfall'].median(),
    'Temp': crop_requirements_df['Temp'].median(),
})
default_area_yield = pd.Series({
    'area_median': crop_area_yield_agg['area_median'].median(),
    'yield_median': crop_area_yield_agg['yield_median'].median(),
    'yield_per_area': crop_area_yield_agg['yield_per_area'].median(),
})

# Function to extract year from filename (if available)
def extract_year_from_filename(crop_name):
    import re
    years = re.findall(r'\b(20\d{2})\b', crop_name)
    if years:
        return int(years[0])
    return 2020

# Build feature matrix - ONE ROW PER TRANSACTION
training_data = []

```



```

crop_list_records = []
year_records = []
target_proxy_records = []

# Get all district soil data for mapping
district_soil_data = soil_summary

print('Building enriched multi-row dataset with HIGH-PERFORMANCE target
    ...')

total_records = 0
req_matches = 0
area_yield_matches = 0

for crop_name, crop_df in crop_data_dict.items():
    base_crop = crop_name.rsplit('-', 1)[0] if any(y in crop_name for y
in ['-2015-2019', '-2019-2022', '-2022-2025', '-2024-2025', '-2025'])
    else crop_name
    crop_key = normalize_crop_name(base_crop)

    file_year = extract_year_from_filename(crop_name)

    req_values = requirements_lookup.loc[crop_key] if crop_key in
requirements_lookup.index else default_req
    ay_values = area_yield_lookup.loc[crop_key] if crop_key in
area_yield_lookup.index else default_area_yield

    if crop_key in requirements_lookup.index:
        req_matches += 1
    if crop_key in area_yield_lookup.index:
        area_yield_matches += 1

    for _, row in crop_df.iterrows():
        year = file_year
        district = row['District Name']

```

```

try:
    if district in district_soil_data.index:
        soil_features = list(district_soil_data.loc[district].
values)
    else:
        soil_features = list(tf_soil.values)
except:
    soil_features = list(tf_soil.values)

req_features = [
    float(req_values['N_Req_level']),
    float(req_values['P_Req_level']),
    float(req_values['K_Req_level']),
    float(req_values['Rainfall']),
    float(req_values['Temp']),
]

ay_features = [
    float(ay_values['area_median']) if not pd.isna(ay_values['
area_median']) else float(default_area_yield['area_median']),
    float(ay_values['yield_median']) if not pd.isna(ay_values['
yield_median']) else float(default_area_yield['yield_median']),
    float(ay_values['yield_per_area']) if not pd.isna(ay_values[
'yield_per_area']) else float(default_area_yield['yield_per_area']),
]

# NO PRICE in model features (deweighting price for suitability
learning)
features = soil_features + list(weather_features.values) +
req_features + ay_features

# Build target proxy from market opportunity: price * crop
historical yield

```

```

        price_value = pd.to_numeric(row.get('Modal Price (Rs./Quintal)',
np.nan), errors='coerce')
        yield_for_proxy = ay_features[1]
        if pd.isna(price_value):
            target_proxy = np.nan
        else:
            target_proxy = float(price_value) * float(yield_for_proxy)

        training_data.append(features)
        crop_list_records.append(base_crop)
        year_records.append(year)
        target_proxy_records.append(target_proxy)
        total_records += 1

# Convert to arrays
X = np.array(training_data)

# Create HIGH-PERFORMING target from top quartile of revenue proxy
proxy_series = pd.Series(target_proxy_records, dtype='float64')
if proxy_series.notna().sum() == 0:
    raise ValueError('No valid price-based target proxy values found.')

proxy_fill_value = float(proxy_series.median())
proxy_series = proxy_series.fillna(proxy_fill_value)
high_perf_threshold = float(proxy_series.quantile(0.75))
y = (proxy_series >= high_perf_threshold).astype(int).to_numpy()

# Create DataFrame for reference
requirement_feature_names = ['req_n_level', 'req_p_level', 'req_k_level',
    , 'req_rainfall', 'req_temp']
area_yield_feature_names = ['hist_area_median', 'hist_yield_median', '
    hist_yield_per_area']
feature_names = list(tf_soil.index) + list(weather_features.index) +
    requirement_feature_names + area_yield_feature_names

```

```

X_df = pd.DataFrame(X, columns=feature_names)
X_df['crop'] = crop_list_records
X_df['success'] = y
X_df['year'] = year_records
X_df['target_revenue_proxy'] = proxy_series.values

# For crop list, get unique crops
crop_list = list(set(crop_list_records))

print('\nDataset created with HIGH-PERFORMANCE target:')
print(f'    Total transaction records: {total_records}')
print(f'    Dataset shape: {X.shape}')
print(f'    Unique crops: {len(crop_list)}')
print(f'    High-performance threshold (75th percentile proxy): {
    high_perf_threshold:.2f}')
print(f'    Classes: {np.bincount(y)}')
print(f'    - High-performing (1): {int(sum(y))} records ({sum(y)/len(y)
    :.1%}')
print(f'    - Lower-performing (0): {int(len(y) - sum(y))} records ({(len
    (y)-sum(y))/len(y):.1%}')
print(f'    Features per record: {len(feature_names)} (PRICE EXCLUDED)')
print('\nExternal feature coverage:')
print(f'    - Crops matched to requirements: {req_matches}/{len(
    crop_data_dict)}')
print(f'    - Crops matched to area/yield: {area_yield_matches}/{len(
    crop_data_dict)}')
print('\nYear distribution (from filenames):')
year_dist = pd.Series(year_records).value_counts().sort_index()
print(year_dist.to_string())

## 6. Model Training
# Train/test split: stratified random split (80/20)
print('Applying STRATIFIED RANDOM split (80% train / 20% test)...')

```

```

X_source = X_df.drop(columns=['year', 'target_revenue_proxy'])

split_idx = np.arange(len(X_df))
train_idx, test_idx = train_test_split(
    split_idx, test_size=0.2, random_state=42, stratify=y
)

y_train = y[train_idx]
y_test = y[test_idx]

print('\nTRAIN/TEST SPLIT RESULTS (Stratified Random):')
print(f'  Train rows (80%): {len(train_idx):,}')
print(f'  Test rows (20%):  {len(test_idx):,}')
print(f'  Train class distribution: {np.bincount(y_train)}')
print(f'  Test class distribution:  {np.bincount(y_test)}')
print(f'  Train high-performance prevalence: {sum(y_train)/len(y_train)
      :.1%}')
print(f'  Test high-performance prevalence: {sum(y_test)/len(y_test)
      :.1%}')

# Define feature sets for modeling (PRICE DEWEIGHTED - only agro-
climatic features)
soil_cols = list(tf_soil.index)
weather_cols = list(weather_features.index)
req_cols = requirement_feature_names
ay_cols = area_yield_feature_names

feature_sets = {
    'soil_weather': soil_cols + weather_cols,
    'soil_weather_requirements': soil_cols + weather_cols + req_cols,
    'soil_weather_req_area_yield': soil_cols + weather_cols + req_cols +
    ay_cols,
}

```

```

print('Feature sets defined (PRICE-DEWEIGHTED for pure agro-climatic
      suitability):')
for set_name, cols in feature_sets.items():
    print(f' - {set_name}: {len(cols)} features')

## 7. Results

# Shared model-training state and helpers

def get_scale_pos_weight():
    if 'y_train' not in globals():
        return 1.0

    neg_count = int((y_train == 0).sum())
    pos_count = int((y_train == 1).sum())
    return (neg_count / pos_count) if pos_count > 0 else 1.0

scale_pos_weight = get_scale_pos_weight()
results = []
trained_models = {}
skipped_runs = []
svm_train_cap = 20000
missing_models = []

# Cross-validation controls
cv_folds = 5
cv_max_rows = 80000

def sample_for_cv(X_data, y_data, max_rows=80000, random_state=42):
    if len(X_data) <= max_rows:
        return X_data, y_data

    rng = np.random.default_rng(random_state)
    pos_idx = np.where(y_data == 1)[0]
    neg_idx = np.where(y_data == 0)[0]

```

```

n_pos = min(len(pos_idx), max_rows // 2)
n_neg = min(len(neg_idx), max_rows - n_pos)

pos_sel = rng.choice(pos_idx, size=n_pos, replace=False)
neg_sel = rng.choice(neg_idx, size=n_neg, replace=False)
sel = np.concatenate([pos_sel, neg_sel])
rng.shuffle(sel)
return X_data[sel], y_data[sel]

def run_cross_validation(model_name, model, X_train_model, y_train_model
):
    if len(np.unique(y_train_model)) < 2:
        return None

    X_cv, y_cv = sample_for_cv(X_train_model, y_train_model, max_rows=
cv_max_rows, random_state=42)

    if len(np.unique(y_cv)) < 2:
        return None

    skf = StratifiedKFold(n_splits=cv_folds, shuffle=True, random_state
=42)
    fold_acc = []
    fold_f1 = []
    fold_pr_auc = []
    fold_roc_auc = []

    for fold_train_idx, fold_val_idx in skf.split(X_cv, y_cv):
        X_fold_train = X_cv[fold_train_idx]
        y_fold_train = y_cv[fold_train_idx]
        X_fold_val = X_cv[fold_val_idx]
        y_fold_val = y_cv[fold_val_idx]

```

```

cv_model = clone(model)

# Keep CV logging quiet for CatBoost
if model_name == 'CatBoost':
    try:
        cv_model.set_params(verbose=False)
    except Exception:
        pass

cv_model.fit(X_fold_train, y_fold_train)
y_val_pred = cv_model.predict(X_fold_val)
y_val_proba = cv_model.predict_proba(X_fold_val)[: , 1]

fold_acc.append(accuracy_score(y_fold_val, y_val_pred))
fold_f1.append(f1_score(y_fold_val, y_val_pred, zero_division=0)
)

fold_pr_auc.append(average_precision_score(y_fold_val,
y_val_proba))

fold_roc_auc.append(roc_auc_score(y_fold_val, y_val_proba))

return {
    'cv_rows': int(len(y_cv)),
    'cv_accuracy_mean': float(np.mean(fold_acc)),
    'cv_accuracy_std': float(np.std(fold_acc)),
    'cv_f1_mean': float(np.mean(fold_f1)),
    'cv_f1_std': float(np.std(fold_f1)),
    'cv_pr_auc_mean': float(np.mean(fold_pr_auc)),
    'cv_pr_auc_std': float(np.std(fold_pr_auc)),
    'cv_roc_auc_mean': float(np.mean(fold_roc_auc)),
    'cv_roc_auc_std': float(np.std(fold_roc_auc)),
}

def train_model_block(model_name, model, feature_set_names=None):
    selected_sets = feature_set_names if feature_set_names is not None

```



```

else list(feature_sets.keys())
print(f'\n=== Starting {model_name} ===')

for feature_set_name in selected_sets:
    cols = feature_sets[feature_set_name]
    print(f'[{model_name}] Preparing feature set: {feature_set_name}
')

X_train_raw = X_source.iloc[train_idx][cols].values
X_test_raw = X_source.iloc[test_idx][cols].values

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_raw)
X_test_scaled = scaler.transform(X_test_raw)

use_scaled = model_name in ['LogisticRegression', 'SVM']
X_train_use = X_train_scaled if use_scaled else X_train_raw
X_test_use = X_test_scaled if use_scaled else X_test_raw

if np.all(np.nanstd(X_train_use, axis=0) < 1e-12):
    message = f'{model_name} on {feature_set_name}: constant
features'
    print(f'[{model_name}] Skipping - {message}')
    skipped_runs.append(message)
    continue

if model_name == 'SVM' and len(X_train_use) > svm_train_cap:
    pos_idx = np.where(y_train == 1)[0]
    neg_idx = np.where(y_train == 0)[0]
    svm_pos_n = min(len(pos_idx), svm_train_cap // 2)
    svm_neg_n = min(len(neg_idx), svm_train_cap - svm_pos_n)
    svm_pos_sel = np.random.choice(pos_idx, size=svm_pos_n,
replace=False)
    svm_neg_sel = np.random.choice(neg_idx, size=svm_neg_n,

```

```

replace=False)

    svm_idx = np.concatenate([svm_pos_sel, svm_neg_sel])
    np.random.shuffle(svm_idx)
    X_train_fit = X_train_use[svm_idx]
    y_train_fit = y_train[svm_idx]
    print(f'[{model_name}] SVM checkpoint: using {len(svm_idx)}
:,{rows} rows for fit')
    else:
        X_train_fit = X_train_use
        y_train_fit = y_train

    print(f'[{model_name}] Running {cv_folds}-fold CV (up to {
cv_max_rows},{rows} rows) ...')
    cv_stats = None
    try:
        cv_stats = run_cross_validation(model_name, model,
X_train_fit, y_train_fit)
        if cv_stats is not None:
            print(
                f"[{model_name}] CV PR-AUC={cv_stats['cv_pr_auc_mean']:.4f} (+/- {cv_stats['cv_pr_auc_std']:.4f}) | "
                f"CV ROC-AUC={cv_stats['cv_roc_auc_mean']:.4f} | CV
F1={cv_stats['cv_f1_mean']:.4f}"
            )
        except Exception as ex:
            message = f"[{model_name}] CV on {feature_set_name}: {str(ex)
[:120]}"
            print(f'[{model_name}] CV failed - {message}')
            skipped_runs.append(message)

    print(f'[{model_name}] Fitting on {len(X_train_fit)}, {rows};
testing on {len(X_test_use)}, {rows}')
    try:
        model.fit(X_train_fit, y_train_fit)

```

```

        y_pred = model.predict(X_test_use)
        y_proba = model.predict_proba(X_test_use)[: , 1]
except Exception as ex:
    message = f"{model_name} on {feature_set_name}: {str(ex)}
[:120]}"

    print(f'[{model_name}] Failed - {message}')
    skipped_runs.append(message)
    continue

accuracy_value = accuracy_score(y_test, y_pred)
f1_value = f1_score(y_test, y_pred, zero_division=0)
pr_auc_value = average_precision_score(y_test, y_proba)
roc_auc_value = roc_auc_score(y_test, y_proba)

row_result = {
    'feature_set': feature_set_name,
    'model': model_name,
    'accuracy': accuracy_value,
    'f1': f1_value,
    'pr_auc': pr_auc_value,
    'roc_auc': roc_auc_value,
    'cv_rows': np.nan,
    'cv_accuracy_mean': np.nan,
    'cv_accuracy_std': np.nan,
    'cv_f1_mean': np.nan,
    'cv_f1_std': np.nan,
    'cv_pr_auc_mean': np.nan,
    'cv_pr_auc_std': np.nan,
    'cv_roc_auc_mean': np.nan,
    'cv_roc_auc_std': np.nan,
}

if cv_stats is not None:
    row_result.update(cv_stats)

```

```

results.append(row_result)

trained_models[(feature_set_name, model_name)] = {
    'model': model,
    'scaler': scaler if use_scaled else None,
    'features': cols,
}

print(
    f'[{model_name}] Done | holdout accuracy={accuracy_value:.4f}
} | ',
    f'holdout f1={f1_value:.4f} | holdout pr_auc={pr_auc_value:.4f} | holdout roc_auc={roc_auc_value:.4f}'
)

print(f'=== Finished {model_name} ===')

def finalize_results():
    global results_df, best_row, best_key, best_pipeline, best_by_model,
    ensemble_members, accuracy_rank
    if len(results) == 0:
        raise ValueError('No model completed training. Check feature
matrix and model availability.')

    results_df = pd.DataFrame(results).sort_values(['pr_auc', 'f1', '
accuracy'], ascending=False).reset_index(drop=True)
    best_row = results_df.iloc[0]
    best_key = (best_row['feature_set'], best_row['model'])
    best_pipeline = trained_models[best_key]

    best_by_model = (
        results_df.sort_values(['pr_auc', 'f1', 'accuracy'], ascending=
False)

```

```

        .groupby('model', as_index=False)
        .first()
        .sort_values('pr_auc', ascending=False)
        .reset_index(drop=True)
    )

    ensemble_members = {}
    for _, row in best_by_model.iterrows():
        member_key = (row['feature_set'], row['model'])
        ensemble_members[row['model']] = trained_models[member_key]

    print('\nModel training completed.')

    if missing_models:
        print(f"Optional libraries missing, skipped models: {'', '.join(
missing_models)}")

    if skipped_runs:
        print('\nSkipped model/feature-set runs:')
        for item in skipped_runs:
            print(f'- {item}')

    print('\nLeaderboard (sorted by holdout PR-AUC):')
    print(results_df[['model', 'feature_set', 'accuracy', 'f1', 'pr_auc',
, 'roc_auc']].to_string(index=False))

    cv_ready = results_df[results_df['cv_pr_auc_mean'].notna()].copy()
    if len(cv_ready) > 0:
        print('\nCross-validation summary (sorted by CV PR-AUC mean):')
        cv_view = cv_ready.sort_values('cv_pr_auc_mean', ascending=False)
    )

    print(cv_view[['model', 'feature_set', 'cv_pr_auc_mean', '
cv_pr_auc_std', 'cv_f1_mean', 'cv_roc_auc_mean', 'cv_rows']].
to_string(index=False))

    print('\nAccuracy ranking for every model (descending):')

```

```

accuracy_rank = results_df.sort_values(['accuracy', 'pr_auc', 'f1'],
    ascending=[False, False, False]).reset_index(drop=True)
print(accuracy_rank[['model', 'feature_set', 'accuracy', 'f1', 'pr_auc', 'roc_auc']].to_string(index=False))

```

```

print('\nBest configuration (by holdout PR-AUC):')
print(f"Feature set: {best_row['feature_set']}")
print(f"Model: {best_row['model']}")
print(
    f"PR-AUC: {best_row['pr_auc']:.4f} | F1: {best_row['f1']:.4f} | "
    f"Accuracy: {best_row['accuracy']:.4f} | ROC-AUC: {best_row['roc_auc']:.4f}"
)

```

```

print('\nEnsemble members (best variant of each model):')
print(best_by_model[['model', 'feature_set', 'pr_auc', 'f1', 'accuracy', 'roc_auc']].to_string(index=False))

```

```

plt.figure(figsize=(14, 6))
plot_df = results_df.head(15).copy()
plot_df['label'] = plot_df['model'] + ' | ' + plot_df['feature_set']
sns.barplot(data=plot_df, y='label', x='pr_auc', palette='viridis')
plt.title('Top 15 Experiments by Holdout PR-AUC')
plt.xlabel('PR-AUC')
plt.ylabel('Model | Feature Set')
plt.tight_layout()
plt.show()

```

Model Cells

Each cell below trains one model family and prints checkpoints so you can track progress as the notebook runs.

```

print('Starting Random Forest training block...')
rf_model = RandomForestClassifier(
    n_estimators=200,

```

```

        max_depth=14,
        min_samples_leaf=2,
        class_weight='balanced_subsample',
        random_state=42,
        n_jobs=-1,
    )

    train_model_block('RandomForest', rf_model)
    print('Random Forest block complete.')
    print('Starting Gradient Boosting training block...')
    gb_model = GradientBoostingClassifier(
        n_estimators=150,
        learning_rate=0.05,
        max_depth=4,
        random_state=42,
    )

    train_model_block('GradientBoosting', gb_model)
    print('Gradient Boosting block complete.')
    print('Starting Logistic Regression training block...')
    lr_model = LogisticRegression(
        max_iter=1200,
        class_weight='balanced',
        solver='lbfgs',
        random_state=42,
    )

    train_model_block('LogisticRegression', lr_model)
    print('Logistic Regression block complete.')
    print('Starting SVM training block...')
    svm_model = SVC(
        C=1.5,
        kernel='rbf',
        gamma='scale',
        class_weight='balanced',
        probability=True,
        random_state=42,

```

```

)
train_model_block('SVM', svm_model)
print('SVM block complete.')
print('Starting CatBoost training block...')
scale_pos_weight = get_scale_pos_weight()
if CatBoostClassifier is not None:
    catboost_model = CatBoostClassifier(
        iterations=260,
        depth=6,
        learning_rate=0.05,
        loss_function='Logloss',
        eval_metric='PRAUC',
        class_weights=[1.0, scale_pos_weight],
        random_seed=42,
        verbose=50,
    )
    train_model_block('CatBoost', catboost_model)
else:
    missing_models.append('CatBoost')
    print('CatBoost is not installed; skipping this block.')
print('CatBoost block complete.')
print('Starting XGBoost training block...')
scale_pos_weight = get_scale_pos_weight()
if XGBClassifier is not None:
    xgb_model = XGBClassifier(
        n_estimators=260,
        max_depth=6,
        learning_rate=0.05,
        subsample=0.9,
        colsample_bytree=0.9,
        reg_lambda=1.0,
        objective='binary:logistic',
        eval_metric='logloss',
        scale_pos_weight=scale_pos_weight,

```



```

        random_state=42,
        n_jobs=-1,
    )
    train_model_block('XGBoost', xgb_model)
else:
    missing_models.append('XGBoost')
    print('XGBoost is not installed; skipping this block.')
print('XGBoost block complete.')
print('Starting LightGBM training block...')
scale_pos_weight = get_scale_pos_weight()
if LGBMClassifier is not None:
    lgbm_model = LGBMClassifier(
        n_estimators=260,
        learning_rate=0.05,
        num_leaves=31,
        max_depth=-1,
        subsample=0.9,
        colsample_bytree=0.9,
        class_weight={0: 1.0, 1: scale_pos_weight},
        random_state=42,
        n_jobs=-1,
        verbose=-1,
    )
    train_model_block('LightGBM', lgbm_model)
else:
    missing_models.append('LightGBM')
    print('LightGBM is not installed; skipping this block.')
print('LightGBM block complete.')
print('Finalizing training results and ensemble members...')
finalize_results()
print('Training finalization complete.')
# Results summary and quick visualization
print('Results summary by feature set (best model per set):')
best_by_set = results_df.sort_values(['pr_auc', 'f1'], ascending=False).

```

```

groupby('feature_set', as_index=False).first()

primary_cols = ['feature_set', 'model', 'pr_auc', 'roc_auc', 'f1', '
    accuracy']
available_primary_cols = [c for c in primary_cols if c in best_by_set.
    columns]
print(best_by_set[available_primary_cols].to_string(index=False))

if 'cv_pr_auc_mean' in results_df.columns:
    cv_ready = results_df[results_df['cv_pr_auc_mean'].notna()].copy()
    if len(cv_ready) > 0:
        print('\nCross-validation leaderboard (CV PR-AUC mean):')
        cv_cols = ['model', 'feature_set', 'cv_pr_auc_mean', '
cv_pr_auc_std', 'cv_f1_mean', 'cv_roc_auc_mean']
        available_cv_cols = [c for c in cv_cols if c in cv_ready.columns
    ]

        cv_view = cv_ready.sort_values('cv_pr_auc_mean', ascending=False
    ).head(10)

        print(cv_view[available_cv_cols].to_string(index=False))

plt.figure(figsize=(12, 5))
plot_df = results_df.head(10).copy()
labels = plot_df['feature_set'] + ' | ' + plot_df['model']
plt.barh(labels[::-1], plot_df['pr_auc'][:, -1], color='#2E8B57')
plt.xlabel('PR-AUC')
plt.title('Top 10 Experiments by Holdout PR-AUC')
plt.tight_layout()
plt.show()

# Classification report for best pipeline
X_test_best = X_source.iloc[test_idx][best_pipeline['features']].values
if best_pipeline['scaler'] is not None:
    X_test_best_use = best_pipeline['scaler'].transform(X_test_best)
else:

```

```

X_test_best_use = X_test_best
y_pred_best = best_pipeline['model'].predict(X_test_best_use)

print('\nClassification report for best pipeline:')
print(classification_report(y_test, y_pred_best, target_names=['Lower
    Performance', 'High Performance']))

## 8. Prediction

# Three prediction rankings: SEPARATE CONCERNS

# 1. HIGH-PERFORMANCE PROBABILITY: Ensemble of non-price features
# 2. YIELD RANK: Historical productivity potential (0-100 normalized)
# 3. PROFIT RANK: Market profitability proxy (Price x Yield)

print('=' * 80)
print('PREDICTION RANKINGS (3 separate objectives):')
print('=' * 80)

# Build ensemble probability from best member of each trained model
    family
ensemble_proba_parts = []
for member_name, member in ensemble_members.items():
    X_member = X_df[member['features']].values
    if member['scaler'] is not None:
        X_member = member['scaler'].transform(X_member)
    member_proba = member['model'].predict_proba(X_member)[: , 1]
    ensemble_proba_parts.append(member_proba)

if len(ensemble_proba_parts) == 0:
    raise ValueError('No ensemble members available. Run training cell
        first.')

ensemble_proba_all = np.mean(np.vstack(ensemble_proba_parts), axis=0)

prediction_df = pd.DataFrame({
    'Crop': X_df['crop'],

```

```

        'Actual_High_Performance': X_df['success'],
        'Ensemble_High_Performance_Probability': ensemble_proba_all,
    })

print('\nRANKING 1: HIGH-PERFORMANCE PROBABILITY (non-price model output
    )')

print('-' * 80)

performance_rank = prediction_df.groupby('Crop', as_index=False).agg(
    Mean_High_Performance_Prob=('Ensemble_High_Performance_Probability',
        'mean'),
    Max_High_Performance_Prob=('Ensemble_High_Performance_Probability',
        'max'),
    Seen_As_High_Performing=('Actual_High_Performance', 'max'),
    Samples=('Crop', 'count'),
).sort_values('Max_High_Performance_Prob', ascending=False)
print(performance_rank.head(15).to_string(index=False))

print('\n\nRANKING 2: YIELD POTENTIAL (Historical productivity, 0-100
    scale)')

print('-' * 80)

yield_rank = crop_area_yield_agg.copy()
yield_rank['Crop'] = yield_rank['crop_key']
yield_rank = yield_rank[['Crop', 'area_median', 'yield_median', '
    yield_per_area']].copy()
yield_rank['Yield_Potential'] = (yield_rank['yield_median'].rank() / len(
    yield_rank)) * 100
yield_rank['Area_Potential'] = (yield_rank['area_median'].rank() / len(
    yield_rank)) * 100
yield_rank['Yield_Combined_Score'] = (0.7 * yield_rank['Yield_Potential'
    ] + 0.3 * yield_rank['Area_Potential'])
yield_rank = yield_rank.sort_values('Yield_Combined_Score', ascending=
    False)
print(yield_rank.head(15).to_string(index=False))

```

```

print('\n\nRANKING 3: PROFIT PROXY (Price x Yield market opportunity)')
print('-' * 80)
profit_lookup = []
for crop_name, crop_df in crop_data_dict.items():
    base_crop = crop_name.rsplit('-', 1)[0] if any(y in crop_name for y
in ['-2015-2019', '-2019-2022', '-2022-2025', '-2024-2025', '-2025'])
    else crop_name
    prices = pd.to_numeric(crop_df['Modal Price (Rs./Quintal)'], errors=
'coerce').dropna()
    if len(prices) == 0:
        continue
    crop_key = normalize_crop_name(base_crop)
    yield_row = crop_area_yield_agg[crop_area_yield_agg['crop_key'] ==
crop_key]
    yield_value = float(yield_row['yield_median'].iloc[0]) if len(
yield_row) > 0 and pd.notna(yield_row['yield_median'].iloc[0]) else
np.nan
    profit_lookup.append({
        'Crop': base_crop,
        'Avg_Price': prices.mean(),
        'Median_Price': prices.median(),
        'Price_Std': prices.std(),
        'Records': len(prices),
        'Yield_Median': yield_value,
    })

profit_rank = pd.DataFrame(profit_lookup)
profit_rank['Gross_Revenue_Proxy'] = profit_rank['Avg_Price'] *
    profit_rank['Yield_Median']
profit_rank['Profit_Proxy_Score'] = (profit_rank['Gross_Revenue_Proxy'].
    rank() / len(profit_rank)) * 100
profit_rank = profit_rank.sort_values('Profit_Proxy_Score', ascending=
    False)
print(profit_rank.head(15).to_string(index=False))

```

```

print('\n' + '=' * 80)
print('INTERPRETATION GUIDE:')
print('=' * 80)
print('')
- HIGH PERFORMANCE PROBABILITY: Model-estimated chance of being top-
    performing (non-price features).
- HIGH YIELD POTENTIAL: Historically productive and scalable crop.
- HIGH PROFIT PROXY: Strong market revenue opportunity (price x yield).

Recommendation: prioritize crops that score high in both performance
    probability and yield,
then use profit proxy as the market viability filter.
'''
print(f"Ensemble members used: {'', '.join(list(ensemble_members.keys()))
    }")

# Keep legacy name for compatibility with previous cells/users
suitability_rank = performance_rank.copy()

## 9. Feature Contribution Analysis

This section explains which input features contribute most to the best-
    performing model and saves a chart for reporting.
from sklearn.inspection import permutation_importance

print('=' * 80)
print('FEATURE CONTRIBUTION ANALYSIS')
print('=' * 80)

if best_pipeline is None:
    raise ValueError('Best pipeline is not available. Run the training
        cells first.')

```

```

feature_names_best = best_pipeline['features']
X_test_features = X_source.iloc[test_idx][feature_names_best].values
if best_pipeline['scaler'] is not None:
    X_test_features = best_pipeline['scaler'].transform(X_test_features)

perm_result = permutation_importance(
    best_pipeline['model'],
    X_test_features,
    y_test,
    n_repeats=10,
    random_state=42,
    scoring='average_precision'
)

importance_df = pd.DataFrame({
    'Feature': feature_names_best,
    'Importance': perm_result.importances_mean,
    'Std': perm_result.importances_std,
})

importance_df = importance_df.sort_values('Importance', ascending=False)
    .reset_index(drop=True)

print('\nTop feature contributions (permutation importance):')
print(importance_df.head(15).to_string(index=False))

feature_groups = {
    'Soil': list(tf_soil.index),
    'Weather': list(weather_features.index),
    'Crop Requirements': requirement_feature_names,
    'Historical Area/Yield': area_yield_feature_names,
}

group_scores = []
for group_name, group_features in feature_groups.items():

```

```

    group_importance = importance_df[importance_df['Feature'].isin(
group_features)][['Importance']].sum()

    group_scores.append({'Group': group_name, 'Importance':
group_importance})

group_df = pd.DataFrame(group_scores).sort_values('Importance',
    ascending=False)

print('\nFeature group contribution summary:')
print(group_df.to_string(index=False))

plt.figure(figsize=(12, 7))
top_features = importance_df.head(12).iloc[::-1]
plt.barh(top_features['Feature'], top_features['Importance'], color='#4
    C78A8', edgecolor='black')
plt.xlabel('Permutation Importance')
plt.ylabel('Feature')
plt.title('Top Feature Contributions for Best Pipeline')
plt.tight_layout()
plt.savefig('feature_contribution_analysis.png', dpi=300, bbox_inches='
    tight')
plt.show()

plt.figure(figsize=(10, 5))
plt.bar(group_df['Group'], group_df['Importance'], color=['#2E8B57', '
    #4682B4', '#C44E52', '#8172B2'], edgecolor='black')
plt.ylabel('Total Importance')
plt.title('Feature Group Contribution Summary')
plt.xticks(rotation=20, ha='right')
plt.tight_layout()
plt.savefig('feature_group_contribution_summary.png', dpi=300,
    bbox_inches='tight')
plt.show()

print('\nSaved images:')

```



```

print(' - feature_contribution_analysis.png')
print(' - feature_group_contribution_summary.png')

## 10. Correlation Matrix Analysis

This section visualizes correlation structure across numeric model
    inputs and shows which features are most correlated with the target
    label.

print('=' * 80)
print('CORRELATION MATRIX ANALYSIS')
print('=' * 80)

# Build numeric analysis table from model dataset
corr_df = X_df.copy()
if 'crop' in corr_df.columns:
    corr_df = corr_df.drop(columns=['crop'])

numeric_corr_df = corr_df.select_dtypes(include=[np.number]).copy()

# Correlation matrix across numeric variables
corr_matrix = numeric_corr_df.corr()

plt.figure(figsize=(16, 12))
sns.heatmap(
    corr_matrix,
    cmap='coolwarm',
    center=0,
    square=True,
    linewidths=0.2,
    cbar_kws={'shrink': 0.8}
)

plt.title('Correlation Matrix of Numeric Model Features', fontsize=14)
plt.tight_layout()
plt.savefig('correlation_matrix_features.png', dpi=300, bbox_inches='
    tight')

```

```

plt.show()

# Correlation of features with target label
if 'success' in numeric_corr_df.columns:
    target_corr = corr_matrix['success'].drop(labels=['success']).
    sort_values(key=lambda s: s.abs(), ascending=False)

    top_target_corr = target_corr.head(15)
    print('\nTop feature correlations with success target:')
    print(top_target_corr.to_string())

    plt.figure(figsize=(12, 7))
    plot_target_corr = top_target_corr.iloc[:::-1]
    colors = ['#2E8B57' if v >= 0 else '#C44E52' for v in
    plot_target_corr.values]
    plt.barh(plot_target_corr.index, plot_target_corr.values, color=
    colors, edgecolor='black')
    plt.axvline(0, color='black', linewidth=1)
    plt.xlabel('Correlation with success')
    plt.ylabel('Feature')
    plt.title('Top Feature Correlations with Target (success)')
    plt.tight_layout()
    plt.savefig('feature_target_correlation.png', dpi=300, bbox_inches='
    tight')
    plt.show()
else:
    print("Target column 'success' not found in numeric data. Skipping
    target-correlation plot.")

print('\nSaved images:')
print(' - correlation_matrix_features.png')
print(' - feature_target_correlation.png')
# Import required libraries
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay,

```

```
classification_report

import matplotlib.pyplot as plt

# Assuming you already have:
# y_test -> actual labels
# y_pred -> predicted labels from your model

# Compute confusion matrix
cm = confusion_matrix(y_test, y_pred)

# Display confusion matrix
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap=plt.cm.Blues)

plt.title("Confusion Matrix")
plt.show()

# Optional: Print detailed classification report
print("Classification Report:\n")
print(classification_report(y_test, y_pred))
```
